

LINGI2364-Project 3: Clustering

Yassine Zeamari - group: 53

May 2026

1 Introduction

The goal of this project is to implement and evaluate two clustering algorithms: K-means and DBSCAN. The objective is to group unlabeled data points into meaningful clusters and compare the behavior of both methods on different datasets.

2 K-means

K-means is a centroid-based clustering algorithm that partitions a dataset into k clusters. The algorithm iteratively alternates between assigning each point to the nearest centroid and updating the centroids as the mean of the assigned points. The objective is to minimize the intra-cluster variance.

2.1 Implementation Choices

Euclidean distance was used to assign each point to its nearest centroid. The centroids were initialized by randomly selecting k observations from the dataset. Since K-means is sensitive to initialization and may converge to local minima, the algorithm was executed multiple times with different random initializations, and the solution with the smallest intra-cluster error was retained.

The stopping criterion was based on the displacement of the centroids between two consecutive iterations. The algorithm stops when this displacement becomes smaller than a threshold ϵ , or when the maximum number of iterations is reached.

2.2 Optimizations

Several optimizations were introduced to improve the robustness and stability of the implementation. First, the algorithm was executed 30 times with different random initializations in order to reduce the probability of converging to poor local minima.

In addition, empty clusters were explicitly handled during the centroid update step. If a cluster became empty, its centroid was reassigned to a randomly

selected data point. This prevents numerical issues and guarantees that exactly k clusters are maintained throughout the execution.

Finally, the convergence criterion avoids unnecessary computations by stopping the algorithm once the centroid positions become sufficiently stable.

3 DBSCAN

DBSCAN is a clustering algorithm based on data density. Unlike K-means, it does not need the number of clusters in advance. The algorithm uses two parameters: ϵ , which defines the neighborhood radius, and MinPts, which defines the minimum number of neighbors needed to create a cluster.

3.1 Implementation Choices

Euclidean distance was used to find the neighbors of each point. A point is considered a core point if it has at least MinPts neighbors inside its ϵ -neighborhood.

The algorithm starts from an unvisited point. If the point is a core point, a new cluster is created and expanded by exploring its neighbors. If a neighbor is also a core point, its neighbors are added to the cluster exploration process.

Three types of labels were used in the implementation:

- -2 for unvisited points,
- -1 for noise points,
- non-negative integers for cluster identifiers.

3.2 Optimizations

For datasets with very large values, the data were standardized before applying DBSCAN. This helps the algorithm use the ϵ parameter more effectively.

An iterative approach was also used to expand clusters. This makes the implementation simpler and avoids recursion problems.

4 Results and Discussion

Dataset	ARI	Silhouette
kmeans_easy_2_4	1.000	0.867
kmeans_hard_4	1.000	0.824
s3_15	0.727	0.492

Table 1: K-means clustering results.

Dataset	ARI	DBI
moons_shape_0.4_5	1.000	–
s2_0.1_10	0.707	0.433
varying_density_1.0_3	1.000	0.298

Table 2: DBSCAN clustering results.

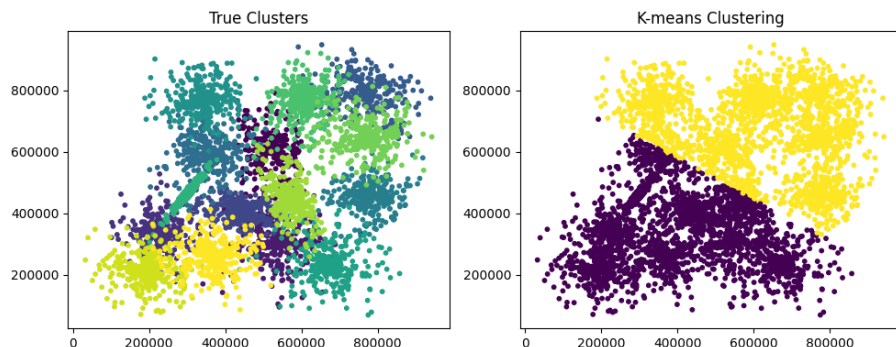


Figure 1: K-means clustering result on the **s3** dataset.

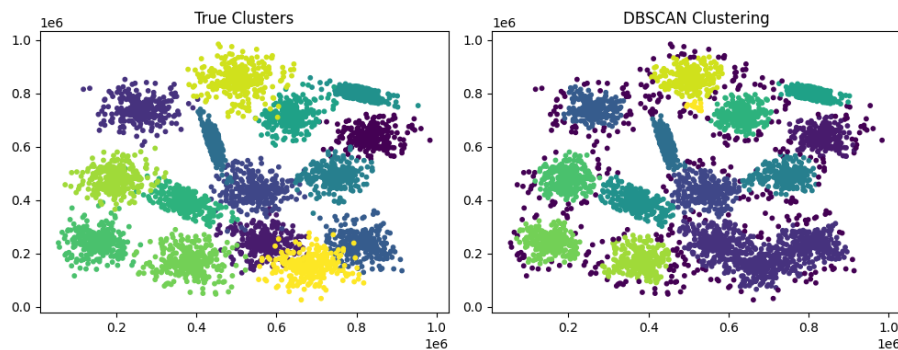


Figure 2: DBSCAN clustering result on the **s2** dataset.

The results show that K-means performs very well on the first two datasets, with an ARI equal to 1. This means that the predicted clusters perfectly match the true labels. The Silhouette scores are also high, showing that the clusters are compact and well separated.

For the **s3** dataset, the ARI decreases to 0.727 and the Silhouette score to 0.492. This suggests that the dataset is more difficult for K-means, and that some clusters are mixed together.

The results also show that DBSCAN performs very well on datasets containing non-spherical clusters and varying densities. For the `moons_shape` and `varying_density` datasets, the algorithm achieved a perfect ARI equal to 1, meaning that the predicted clusters perfectly match the true labels.

For the `s2` dataset, the ARI decreased to approximately 0.71, which indicates that the clustering task was more difficult. However, the result remains acceptable.

The Davies–Bouldin Index (DBI) was also used to evaluate clustering quality. Lower DBI values indicate more compact and better separated clusters. The obtained DBI values confirm that DBSCAN produced good clustering results on most datasets.

The DBI could not be computed for some datasets because only one valid cluster remained after removing noise points.

Figure 1 shows the result of K-means on the `s3` dataset. The algorithm found the general structure of the data, but some clusters were mixed together, which explains the lower ARI value.

Figure 2 shows the result of DBSCAN on the `s2` dataset. The algorithm correctly detected most clusters and handled the complex shapes of the data.

5 Conclusion

In this project, K-means and DBSCAN were successfully implemented and tested on several datasets. The results showed that K-means works very well on compact and well-separated clusters, while DBSCAN is more effective for complex cluster shapes and noisy data.